

Competitive Programming

Section 60 Editorial: Final Capstone

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Read before code: problem dossiers	3
1.2	Problem-specific proof map	3
1.3	Detailed solution guide	3
2	A. Capstone Subset	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
3	B. Capstone CRT	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	7
4	C. Capstone Recurrence	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	9
5	D. Coupon Route	11
5.1	State expansion	11
5.2	Correctness proof	11
5.3	Complexity and tests	11
5.4	Full C++ solution	11
5.5	Full Python solution	11
6	What to carry forward	13

1 Editorial overview

1.1 Read before code: problem dossiers

A. Capstone Subset. Split items in half. Every full subset is one pair of half-subsets; sorted right sums let each left sum find its best legal partner. Test empty subset, exact limit, and all items above the limit.

B. Capstone CRT. Maintain one residue/modulus pair for processed equations. Compatibility is residue agreement modulo the gcd; extended Euclid constructs the merged pair. Test compatible non-coprime moduli and an impossible pair.

C. Capstone Recurrence. The adjacent Fibonacci pair is the invariant; doubling identities process index bits. Test zero and values around a power of two. Each query is $O(\log n)$.

1.2 Problem-specific proof map

A. Capstone Subset. Every subset splits uniquely into a left-half and a right-half subset. Sorting right sums and taking the greatest compatible one for every left sum examines the optimum in $O(2^{\{n/2\}} \log 2^{\{n/2\}})$ time.

B. Capstone CRT. The current pair (a, m) represents exactly all solutions to processed congruences. A new congruence is compatible iff residues agree modulo $\gcd(m, n)$; extended Euclid then constructs its merged residue.

C. Capstone Recurrence. The fast-doubling invariant stores adjacent Fibonacci values. The identities preserve it when a binary digit doubles or doubles-plus-one the index, yielding logarithmic queries.

1.3 Detailed solution guide

A. Capstone Subset Optimization. Split the items into two halves, enumerate all subset sums of each half, and sort the right-half sums. For every left sum, binary-search the best right partner that keeps the total under the limit. This is the meet-in-the-middle proof from Section 47: every full subset has exactly one left subset and one right subset.

B. Capstone Congruences. Merge congruences one at a time with CRT. At each step, the current pair (a, m) represents all numbers satisfying the processed constraints. Merging with (b, n) either detects incompatible residues modulo $\gcd(m, n)$ or produces a new residue modulo $\text{lcm}(m, n)$ representing the intersection. This invariant makes the final answer immediate.

C. Capstone Recurrence. Use fast doubling to evaluate the recurrence in logarithmic time. The state pair stores adjacent Fibonacci values, the doubling formula advances from k to $2k$, and the optional step advances to $2k+1$. Processing all bits of n therefore computes the requested value exactly.

Common pitfalls. The final capstone intentionally mixes techniques. Write the invariant for each module separately, test it locally, then combine the outputs only after each module is correct on its own.

The capstone revisits three patterns that should now be automatic.

- A uses meet-in-the-middle for maximum subset sum under a limit.
- B merges congruences by generalized CRT and rejects incompatible residues.
- C answers recurrence queries with fast Fibonacci doubling.

The final proof style is concise but complete: define the compressed object, prove it preserves the original choices, and show every query reads the desired value from that object.

2 A. Capstone Subset

2.1 Restatement

Meet-in-the-middle: maximum subset sum not exceeding X .

2.2 Observations and algorithm

Split the array into two halves and enumerate all subset sums of each. Sort the right sums; for each left sum binary-search the greatest right partner not exceeding $X - \text{left}$. Trace an empty right subset and an exact-limit pair.

2.3 Correctness proof

Every full subset decomposes uniquely into one subset of each half. The binary search considers the best feasible partner for each left choice, including the optimal decomposition. $\square$

2.4 Complexity

Time $O(2^{(n/2)} \log 2^{(n/2)})$, memory $O(2^{(n/2)})$. Use `long long` sums; test the empty subset and values larger than the limit.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

2.6 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
```

```
if not data: return
n,x=data[0],data[1]; a=data[2:]; left=sums(a[:n//2]); right=sorted(sums(a[n//2:])); best
for s in left:
    i=bisect.bisect_right(right,x-s)-1
    if i>=0: best=max(best,s+right[i])
print(best)
if __name__=="__main__": main()
```

3 B. Capstone CRT

3.1 Restatement

Merge pairs of congruences or report impossibility.

3.2 Observations and algorithm

Maintain $x \equiv a \pmod{m}$ for processed equations. To merge $b \pmod{n}$, first require $(b-a)$ divisible by $\gcd(m,n)$, then use extended Euclid to solve the reduced multiplier and normalize modulo the lcm.

3.3 Correctness proof

The divisibility condition is necessary because both residues must agree modulo the gcd; the constructed multiplier satisfies both equations, so the merged class represents exactly their intersection. $\square$

3.4 Complexity

Extended Euclid costs $O(\log \min(m,n))$. Use wide multiplication for the lcm; test compatible non-coprime moduli and incompatible residues.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; long long eg(long long a,long long b,long long&x,long long&y){if(!b){x=
```

3.6 Full Python solution

```
import sys,math
def eg(a,b):
    if b==0: return (abs(a),1 if a>=0 else -1,0)
    g,x,y=eg(b,a%b); return g,y,x-(a//b)*y
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
```

```

if not data: return
t=data[0]; idx=1; out=[]
for _ in range(t):
    a,m,b,n=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; g,x,y=eg(m,n)
    if (b-a)%g: out.append("NO")
    else:
        l=m//g*n; k=((b-a)//g*x)%(n//g); out.append(f"{(a+m*k)%l} {l}")
print("\n".join(out))
if __name__=="__main__": main()

```

4 C. Capstone Recurrence

4.1 Restatement

Fast Fibonacci queries modulo $1e9+7$.

4.2 Observations and algorithm

Use fast doubling with state $(F(k), F(k+1))$. From it compute values at $2k$, then choose the even or odd pair according to the current index bit.

4.3 Correctness proof

The doubling identities are algebraic Fibonacci identities; induction on the halved index proves the state remains adjacent Fibonacci values. $\square$

4.4 Complexity

$O(\log n)$ time and recursion depth per query. Test 0, 1, and a large power-of-two boundary.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

4.6 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
def main():
```

```
data=list(map(int,sys.stdin.buffer.read().split()))
if not data: return
print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

5 D. Coupon Route

5.1 State expansion

The free-edge coupon is part of the route history. Reaching the same vertex before and after using it are different situations, so use states $(\text{vertex}, \text{used})$ where used is zero or one. From (v, used) , an edge of weight w gives (to, used) at cost w ; when $\text{used}=0$, it also gives $(to, 1)$ at cost zero.

All expanded-edge costs are nonnegative, so run Dijkstra from $(1, 0)$. The answer is the smaller distance to $(n, 0)$ and $(n, 1)$.

5.2 Correctness proof

Every legal original route maps to exactly one expanded path by marking whether its coupon has already been used. Its expanded cost equals its paid cost. Conversely, every expanded path uses at most one zero-cost coupon transition, so it maps to a legal route of the same cost. Dijkstra returns shortest paths in this nonnegative expanded graph; therefore the reported minimum is optimal.

5.3 Complexity and tests

The expanded graph has $2n$ states and at most $4m$ directed transitions, so Dijkstra costs $O((n+m) \log n)$ time and $O(n+m)$ memory. Test one edge (answer zero), a route where the most expensive edge is not on the ordinary shortest path, and a long chain. The tiny oracle uses repeated edge relaxation on both coupon layers; the scale case is a 200000-vertex chain.

5.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false); cin.tie(nullptr); int n,m; if(!(cin
```

5.5 Full Python solution

```

import sys,heapq
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,m=d[:2];g=[[] for _ in range(n)];i=2
    for _ in range(m):
        a,b,w=d[i]-1,d[i+1]-1,d[i+2];i+=3;g[a].append((b,w));g[b].append((a,w))
    inf=10**30;dist=[[inf]*n for _ in range(2)];dist[0][0]=0;pq=[(0,0,0)]
    while pq:
        cost,used,v=heapq.heappop(pq)
        if cost!=dist[used][v]:continue
        for to,w in g[v]:
            if cost+w<dist[used][to]:dist[used][to]=cost+w;heapq.heappush(pq,(cost+w,used,to))
            if not used and cost<dist[1][to]:dist[1][to]=cost;heapq.heappush(pq,(cost,1,to))
    print(min(dist[0][-1],dist[1][-1]))
if __name__=='__main__':main()

```

6 What to carry forward